

The Heroku agent at <https://argon-bd8888db5430.herokuapp.com> already returns the new forecast, pool, and gate data. The Vercel app needs to read those fields and stop treating a missing on-chain transaction as an empty screen.

Use `cache: "no-store"` on every agent fetch. Poll `/forecasts/latest` and `/status` every 30 seconds, and `/pools` every 15 seconds. `txHash: null` and `onchainForecastCount: 0` only mean the keeper is in dry-run. The forecast and pool numbers are still valid.

Forecast hero

`GET /forecasts/latest` returns the current hour when it exists, otherwise the last stored hour with `"live": false`. Do not show "no forecast" on a 200.

Display `predEthUsd8h` as the 8-hour predicted ETH price, and keep `ethPct8h` as the percent. `barCloseUsd` is the candle that price was measured from. `expectedEthUsd1h` is where the averaged 1-hour path says the price should be next.

`ethPct1h` and `ethPct2h` are no longer the raw model output. They are the average remaining move, from the current price, across every stored 8-hour price target that still covers that horizon. `ethPct8h` is only the target issued this hour. If `ethPct1hSource` or `ethPct2hSource` is `"residual"`, label it as the updated remainder of earlier 8-hour prices.

When `live` is false, show the last hour and a stale banner. Keep rendering the percent and the predicted price.

Warmup bar

Read `GET /status`.

UI	Field
Hours filled	<code>9 - hoursUntilFirstDecision</code>
First decision countdown	<code>hoursUntilFirstDecision</code>
On-chain submits	<code>onchainForecastCount</code>

`dbForecastCount` is how many inferences are stored. The bar must use `hoursUntilFirstDecision`, not `onchainForecastCount`. Dry-run stays at 0 on-chain submits while the bar can already be full.

Pool cards

GET /pools returns pools[]. Match id of "arbitrum" or "robinhood".

Card label	Field
APR	aprPct
Pool TVL	poolTvlUsd
ETH	ethUsd

These three lines do not require a wallet. Arbitrum APR comes from DefiLlama (aprSource: "defillama"). Robinhood APR is the fixed Uniswap figure 35.51% (aprSource: "uniswap"). If aprPct is null, show an empty APR and still show TVL and ETH.

Per-user gate

On the pool card, after the user picks Arbitrum or Robinhood, show Safe, Balanced, and Aggressive, plus Custom with a top (+) and a bottom (-).

Preset	1h	2h	8h
Safe	±0.6% (±60 bps)	±1.2%	±1.0%
Balanced	±1.0% (±100 bps)	±2.5%	±2.0%
Aggressive	±2.0% (±200 bps)	±4.0%	±3.5%

Custom sends the user's 1-hour top and bottom in basis points. Top must be positive, bottom negative, each within 20%. The 2-hour and 8-hour bands stay Balanced.

Save only after the connected wallet signs this exact string:

```
argon-gate:{addressLowercase}:{preset}:{topBps}:{bottomBps}:{issuedAt}
```

issuedAt is a Unix second, and it must be within 2 hours of now and newer than the stored gate. personal_sign that string, then:

POST /gates

```
{  
  
  "address": "0x...",  
  
  "preset": "safe",  
  
  "topBps": 60,  
}
```

```
"bottomBps": -60,  
  
"issuedAt": 1710000000,  
  
"signature": "0x..."  
}
```

For Safe, `topBps` must be 60 and `bottomBps` must be -60. Balanced is 100 and -100. Aggressive is 200 and -200. A mismatch returns 400.

GET `/gates/{address}` returns that signer's row: the six bands, `inPosition`, `lastAction` (`warmup`, `enter`, `hold`, or `exit`), and `lastHourId`. A 404 means they have not chosen a gate yet. Show `lastAction` on the card. It is that wallet's decision. The forecast `action` field is still the shared vault action.

Copy on the control: the model still publishes one 8-hour ETH price. The user's gate decides whether their capital is allowed into the Uniswap position that hour.